

Layer	Primary Components	Responsibility	Key Output
Ingress API	Webhook Controller, Signal Controller	Accept webhook events and downstream completion or cancellation signals	Validated requests routed into orchestration flow
Application Services	Webhook Service, Orchestration Service, Queue Producer	Apply business rules, create workflow IDs, dispatch queue messages, control workflow lifecycle	Workflow initiation and state transitions
Workflow Runtime	Temporal Workflow, Temporal Activities, Temporal Worker, Temporal Client Repository	Execute long-running orchestration with retries, timeouts, and signal waiting	Durable workflow execution
Persistence	Workflow Repository, MongoDB collections	Store workflow status and activity execution details	Operational traceability and recovery context
Integration	Azure Service Bus, DataRepo API, Azure Blob, Auth modules	Communicate with external systems and fetch file/site context	Task dispatch and metadata enrichment

Component Design

API Layer

- **POST /api/webhook:** receives one or more upload events from ESDT Portal
- **POST /api/signal/activity:** receives completion or failure notification from downstream services
- **POST /api/signal/cancel:** terminates an active workflow

Webhook Module

The webhook module validates the incoming payload and converts accepted events into workflow initiation requests.

- Confirms source is `e57`
- Confirms required identifiers exist: `fileId`, `customerId`, `siteId`
- Calls orchestration service for each valid event
- Returns per-event success or failure response

Queue Module

The queue module is responsible for emitting orchestration and downstream task messages and processing inbound system messages when used as an internal event router.

- **queue.producer.ts**: sends messages to Azure Service Bus
- **queue.processor.ts**: routes received messages by type
- **orchestration.service.ts**: maps application events to workflow actions
- **workflow.repository.ts**: persists workflow and activity state

Temporal Module

The Temporal module provides durable orchestration. The workflow governs sequence, waiting logic, and timeout behavior. Activities handle side effects such as API calls and queue dispatch.

- **Workflow**: `Scan2BimWorkflow`
- **Activities**: generate SAS URL, start segmentation, register post-processing stage
- **Worker**: polls Temporal task queue and executes activities
- **Client repository**: starts, signals, and terminates workflows

End-to-End Function Flow

1. Webhook Reception

1. ESDT Portal uploads an E57 file and sends a webhook to `/api/webhook`.
2. Webhook controller accepts the payload of events.
3. Webhook service validates each event.
4. For valid events, the service calls `initiateWorkflow(event)`.

 Recommended behavior: treat events independently so that one invalid event does not block valid events in the same request.

2. Workflow Initiation

1. Orchestration service generates a unique workflow ID, for example `scan2bim-<uuid>`.
2. A root queue message is produced with type `WEBHOOK_EVENT` and step `INITIATE_WORKFLOW`.
3. An initial `workflow_executions` record is created with status such as `RECEIVED` or `INITIATED`.
4. Queue processor receives the message and starts the Temporal workflow.

3. Temporal Workflow Execution

The workflow coordinates three main stages.

1. Generate SAS URL

Activity calls DataRepo API to obtain a signed download URL and file details.

2. Dispatch Segmentation

Activity enriches payload with site metadata and sends a task to `esdt-s2b-segmentation-queue`.

3. Wait for Segmentation Signal

Workflow pauses until a signal with activity name `SEGMENTATION` is received or timeout occurs.

4. Register Post-Processing Stage

Activity records the next stage in MongoDB and prepares the workflow to await final completion.

5. Wait for Post-Processing Signal

Workflow pauses until a signal with activity name `POST_PROCESSING` is received or timeout occurs.

6. Complete Workflow

Workflow marks final state and persists completion metadata.

4. Downstream Signal Reception

1. Segmentation service completes its work and calls `/api/signal/activity` with status `COMPLETED` or `FAILED`.
2. Signal controller validates the workflow ID, activity name, and status.
3. The controller forwards the signal into the Temporal workflow.
4. MongoDB activity and workflow records are updated.
5. The same pattern is repeated for Post-Processing completion.

5. Cancellation Flow

1. An external caller sends `/api/signal/cancel` with workflow ID and reason.
2. The Root Microservice invokes Temporal termination.
3. The workflow record is updated to `CANCELLED`.
4. Any active waiting state is ended immediately.

Detailed Sequence Design

Primary happy path

1. Webhook received from ESDT Portal
2. Event validated
3. Workflow ID created
4. Root queue event published
5. Temporal workflow started
6. SAS URL generated from DataRepo
7. Segmentation task published to segmentation queue
8. Workflow waits for segmentation signal
9. Segmentation service processes file and signals completion
10. Workflow records segmentation completion
11. Post-processing stage registered
12. Workflow waits for post-processing signal
13. Post-processing service signals completion
14. Workflow marked completed in Temporal and MongoDB

Failure path

- If webhook validation fails, the event is returned in the failure list and no workflow is started.
- If SAS URL generation fails, Temporal activity retry policy is applied.
- If segmentation dispatch fails, activity retry policy is applied before workflow failure.
- If no signal is received within 4 hours, the workflow times out and status is updated to failed or timed out.
- If a downstream service reports `FAILED`, the signal should update activity state and the workflow should transition to failed.

Internal Contracts

Webhook input contract

Expected minimum fields in each event:

- `source`
- `fileId`
- `customerId`
- `siteId`
- Optional project and metadata fields if provided upstream

Signal input contract

Field	Description	Constraint
<code>workflowId</code>	Unique Temporal workflow identifier	Required
<code>activityName</code>	Workflow stage being completed	<code>SEGMENTATION</code> or <code>POST_PROCESSING</code>
<code>status</code>	Result of downstream stage	<code>COMPLETED</code> or <code>FAILED</code>
<code>payload</code>	Optional result or error metadata	Structured JSON object

Segmentation queue message

The Root Microservice produces a task message for the segmentation queue with traceability and enriched file and site metadata.

- `traceId`
- `workflowId`
- `taskId`
- `payload.customerId`
- `payload.siteId`
- `payload.projectId`
- `payload.fileId`
- `payload.fileUrl`

- `payload.siteName`, `siteType`, `siteElevation`, `latitude`, `longitude`
- `payload.metadata`
- `timestamp`

Persistence Design

workflow_executions

Stores workflow-level state for reporting, troubleshooting, and reconciliation.

- `workflowId`
- `eventId`
- `status`
- `currentActivity`
- `customerId`
- `siteId`
- `fileId`
- `category`
- `createdAt`, `updatedAt`, `completedAt`
- `error`

activity_executions

Stores activity-level execution details and supports auditability of retries and outputs.

- `activityId`
- `workflowId`
- `activityName`
- `status`
- `input`
- `output`
- `error`
- `startedAt`, `completedAt`
- `retryCount`

📌 Recommended statuses for consistency: `RECEIVED`, `IN_PROGRESS`, `WAITING_FOR_SEGMENTATION`, `WAITING_FOR_POST_PROCESSING`, `COMPLETED`, `FAILED`, `CANCELLED`, `TIMED_OUT`.

Suggested LLD Behavior by Module

Webhook Controller

- Accept batch event payload
- Delegate all business validation to service layer
- Return structured response with successes and failures

Webhook Service

- Validate event source and required identifiers
- Normalize payload shape
- Call orchestration service once per valid event
- Capture validation and initiation errors per event

Orchestration Service

- Create workflow ID and correlation IDs
- Persist initial workflow record
- Send internal message to root queue or invoke Temporal client directly
- Update workflow status on lifecycle changes

Queue Processor

- Route by message type
- Handle `WEBHOOK_EVENT` , `SEGMENTATION_COMPLETE` , and `POST_PROCESSING_AND_UPLOAD_COMPLETE`
- Ensure idempotent processing using workflow ID and task ID

Temporal Workflow

- Encapsulate orchestration state machine
- Call activities in sequence
- Wait for signals with a 4-hour timeout per downstream stage
- Fail fast on invalid signal or terminal downstream failure

Temporal Activities

- `generateSasUrlActivity` : fetch signed URL from DataRepo
- `startSegmentationActivity` : publish enriched segmentation task
- `startPostProcessingAndUploadActivity` : register activity state and prepare completion tracking

State Transition Model

Current State	Trigger	Next State
RECEIVED	Webhook accepted	INITIATED
INITIATED	Temporal workflow started	IN_PROGRESS
IN_PROGRESS	Segmentation task dispatched	WAITING_FOR_SEGM ENTATION
WAITING_FOR_SEGM ENTATION	Segmentation completed	WAITING_FOR_POST_ PROCESSING
WAITING_FOR_SEGM ENTATION	Segmentation failed or timed out	FAILED
WAITING_FOR_POST_ PROCESSING	Post-processing completed	COMPLETED
WAITING_FOR_POST_ PROCESSING	Post-processing failed or timed out	FAILED

Non-Functional Design Considerations

- **Idempotency:** duplicate webhook or signal calls should not create duplicate active workflows for the same event without explicit intent.
- **Observability:** include `traceId`, `workflowId`, and `taskId` in all logs and queue messages.
- **Retry behavior:** activity retries use exponential backoff of 1s, 2s, and 4s with a 30-minute timeout.
- **Timeout handling:** workflow waiting stages expire after 4 hours and should update persisted state clearly.
- **Security:** protect webhook and signal endpoints with appropriate authentication or signature validation if external exposure exists.
- **Recovery:** MongoDB records should be sufficient to inspect incomplete workflows and reconcile against Temporal state.

✓ Design recommendation: use Temporal as the source of orchestration truth and MongoDB as the operational reporting store. This keeps workflow control durable while still allowing

fast search and support visibility.

Concise Function Flow Summary

Webhook in → validation → workflow creation → Temporal starts → SAS URL fetched → segmentation queue message sent → segmentation signal received → post-processing tracked → post-processing signal received → workflow completed.

Optional implementation notes

For cleaner separation, the Root Microservice can keep all external side effects inside Temporal activities and use the workflow only for sequencing and waiting logic. This reduces workflow complexity and improves testability. Queue payload versioning is also recommended so downstream services can evolve safely without breaking existing contracts.